# 1 Introdução à gestão da engenharia de software

Semana 1 · Resumo da semana

## Ideias-chave, lição por lição

### 1 O que é engenharia de software

- A engenharia de software é uma disciplina sobre entregar e manter software, não apenas escrevê-lo.
- A maioria dos problemas de projeto vem de falhas de comunicação e de planejamento.
- Os modelos permitem que as equipes discutam e verifiquem um sistema antes de construí-lo.

### 2 O ciclo de vida de desenvolvimento e os modelos de processo

- Um modelo de processo é uma descrição simplificada de como equipes reais trabalham.
- O trabalho iterativo produz linhas de base; a diferença entre linhas de base é um incremento.
- Escolha um processo pelo quanto de mudança você espera e pelo custo dos erros tardios.

### 3 Engenharia de requisitos: de desejos a especificações

- Um requisito precisa ser verificável: alguém deve poder dizer "atendido" ou "não atendido".
- Sete atividades: concepção, elicitação, elaboração, negociação, especificação, validação e gestão.
- Funcional = o que o sistema faz. Não funcional = quão bem ele faz.

### 4 Modelar software: de diagramas de contexto à UML

- Um modelo é útil quando é mais simples que o sistema, mas continua correto no que importa.
- Diagrama de contexto = limites; modelo ER = dados; máquina de estados = ciclo de vida de uma coisa.
- A UML tem diagramas de estrutura e de comportamento; escolha pela pergunta que você precisa responder.

### 5 Gestão de configuração de software

- Os itens de configuração incluem tudo o que compõe o produto, não apenas o código.
- A GCS responde: qual versão, qual mudança, aprovada por quem e em qual linha de base.

### 6 Sistemas de controle de versões

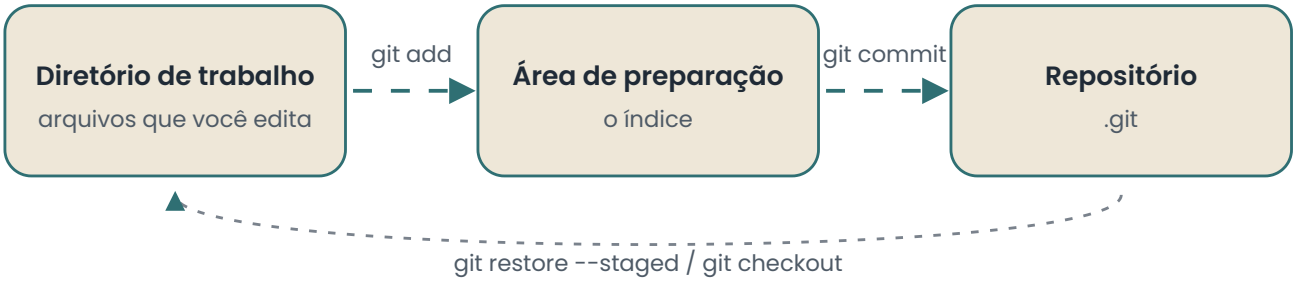
- Um VCS guarda o histórico, permite recuperar versões e mostra quem mudou o quê e quando.
- O Git é distribuído; o GitHub é um serviço de hospedagem de repositórios Git.
- Um commit deve conter uma única mudança lógica com uma mensagem clara.

### 7 Fundamentos do Git: áreas, commits e branches

- Edite no diretório de trabalho, prepare com git add e registre com git commit.
- git status e git log mostram onde você está; use-os antes e depois de cada passo.
- Uma branch é um ponteiro. O merge combina históricos; conflitos exigem uma decisão humana.

### 8 GitHub, remotos e fluxos de colaboração

- origin é o nome convencional do remoto que você clonou ou conectou.
- fetch baixa; pull baixa e integra; push envia.
- Os pull requests tornam a revisão uma etapa planejada antes de o código chegar à main.




## Cuidado com isto

| Erro | Solução |
|---|---|
| Confundir Git com GitHub. | O Git é a ferramenta no seu computador; o GitHub hospeda repositórios. |
| Esquecer de ler o git status antes de fazer commit. | Faça do git status um hábito antes e depois de cada commit. |
| Tratar "compila" como "atende à necessidade". | Compare o resultado com requisitos escritos e testes. |
| Chamar um projeto em cascata de "ágil" porque ele tem reuniões. | Pergunte se o feedback muda o plano durante o projeto. |
| Escrever "o sistema deve ser rápido e fácil". | Acrescente uma medida: "os resultados da busca aparecem em menos de 1 segundo". |
| Mudar requisitos pelo chat sem registro. | Registre cada acordo e quem o aprovou; isso é gestão de requisitos. |
| Desenhar todos os diagramas UML "para ficar completo". | Desenhe o diagrama que responde à pergunta atual. |
| Escrever mensagens de commit como "coisas" ou "mudanças". | Descreva a mudança em uma linha imperativa de cerca de 50 caracteres. |